C 8205 – Testhandbuch

Wie ein Test aufgebaut ist, wie er abläuft und wie man sein Ergebnis liest

Dieses Dokument beschreibt das Testkonzept des Projekts: Jeder simulierte „Rechnerlauf“ ist ein Test, unabhängig davon, ob ihn der Python-Interpreter oder eine KI (Claude-Interpreter) ausführt. Ein Test besteht aus einer Trommeldatei (Speicherinhalt), einer Tastenfolge (Bedienhandlungen) und optional Prüfungen (Sollwerte).

Quelle: C8205-Interpreter-Durchführung_eines_Tests.md (Stand 04.10.2026), ergänzt um die Beobachtungen aus Läufen mit c8205_interp.py (Fassung vom 04.10.2026). Alle gezeigten Protokollauszüge sind echte Läufe.

Hinweis Abgrenzung: Wie man den Interpreter aufruft (Schalter, Fehlermeldungen), steht im PDF „Bedienungsanleitung Python-Interpreter“. Dieses Handbuch erklärt, was ein Test ist und warum er so aufgebaut ist; für Rechner und Bedienpult siehe „Rechnerbeschreibung“ und „Bedienpult und Peripherie“.

Achtung Grundsatz „laut scheitern“: Bei einer Unklarheit, einem Widerspruch oder einer fehlenden Festlegung wird der Test angehalten und das gemeldet – es wird nichts stillschweigend angenommen oder repariert.

Legende

Zeichen	Bedeutung
Trommeldatei, Tastenfolge, Prüfungen	die drei Teile einer Testdatei (Kapitel 2)
Testmarke	Zeile mit #, die nur für den Test gilt und in der Hardware keine Entsprechung hat
Python-/Claude-Interpreter	die beiden Ausführenden desselben Testformats (Kapitel 9)
Hinweis / Achtung	blauer bzw. orangefarbener Balken

Inhaltsverzeichnis

1 Einordnung und Voraussetzungen . . . 3
1.1 Zwei Ausführende, ein Format . . . 3
1.2 Ein Test im Überblick . . . 3
2 Aufbau einer Testdatei . . . 4
3 Schritt 1: die Trommeldatei . . . 4
3.1 Zeilenformat . . . 4
3.2 Testmarken . . . 4
3.3 Testauftrag aus einem Assemblerlauf . . . 5
3.4 Geräte-Zeilen --leser1, --leser2, --stanzer1, --sw-ausgabe und --k0 . . . 5
4 Schritt 2: die Tastenfolge . . . 6
4.1 Beispiel A: Ansprung des Speicherplatzes 0000 (direkt) . . . 6
4.2 Beispiel B: Ansprung eines beliebigen Speicherplatzes (hier 0004) . . . 6
4.3 Festlegungen zu den Tasten M, Ü, E, ST, BS, SP . . . 7
4.4 Wo der Lauf nach „R“ beginnt . . . 7
4.5 Weitere typische Bedienfolgen (Fälle 1 bis 6) . . . 8
5 Schritt 3 (optional): Prüfungen . . . 8
5.1 Aufbau . . . 8
5.2 Auswertung und Ausgabe . . . 9
5.3 Typische Sollwerte . . . 9
5.4 Dateiprüfungen für Stanzer und Schreibwerk . . . 9
6 Formale Gültigkeitsprüfung vor dem Start . . . 10
7 Diagnose beim Leerbefehl 164 . . . 11
7.1 Auslösebedingung . . . 11
7.2 Ablauf . . . 11
8 Protokoll und Protokolltiefe . . . 11
8.1 Inhalt der Protokolldatei (Stufe 2) . . . 12
8.2 Eine Knotenzeile lesen . . . 12
9 Ein- und Ausgabe in Tests . . . 12
9.1 Lochstreifendateien . . . 13
9.2 Schreibwerk . . . 13
9.3 Gegenüberstellung der Ausführenden . . . 13
10 Den Claude-Interpreter einsetzen . . . 13
10.1 Wann? . . . 13
10.2 Was die KI braucht . . . 14
10.3 Vorgehen . . . 14
11 Referenztests und Regression . . . 14
12 Checkliste und Vorlage . . . 14
12.1 Checkliste für neue Tests . . . 14
12.2 Vorlage . . . 15
Anhang A: Begriffe . . . 15
Anhang B: Verweise . . . 15

1 Einordnung und Voraussetzungen

Das Gesamtprojekt beschreibt zwei Aspekte: erstens die Hardware und die in der Elektronik angelegten Beziehungen (Knotenautomat), zweitens davon abweichend oder ergänzend die Annahmen für die interpretative Abarbeitung eines Befehls. Der zweite Aspekt ist die Voraussetzung jedes Tests.

- Alle Zeiten sind 0. Zeitliche Abhängigkeiten werden in ihrer Reihenfolge betrachtet, führen aber nicht zu Wartezeiten.
- Der Rechner ist eingeschaltet. Jeder Test setzt das voraus.
- Der Vor-Zustand vom Einschalten bis zum Beginn der Interpretation der Befehlsschleife am Konnektor B (Knoten 1 des Graphen) ist vollständig bestimmt durch die betätigten Tasten und Schalter und den Trommelinhalt. Mehr gibt es nicht: Alles andere (z. B. Register, die der Test nicht nennt) startet mit 0.

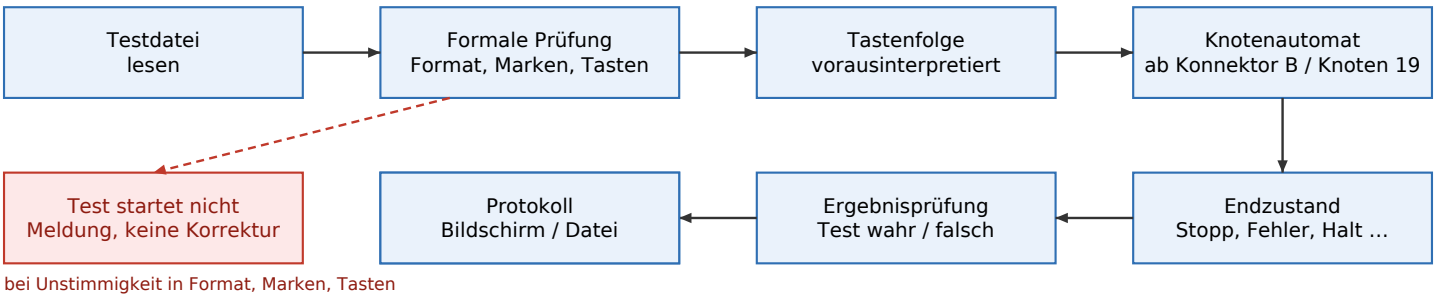
1.1 Zwei Ausführende, ein Format

Ein und derselbe Test kann auf zwei Arten ausgeführt werden. Beide benutzen denselben Knotengraphen, dasselbe Testformat und dasselbe Protokollformat.

	Python-Interpreter	**Claude-Interpreter (KI)**
Wie	c8205_interp.py läuft den Knotengraphen deterministisch und automatisiert ab	Eine KI erhält das komplette Regelwerk als Kontext und die Testdatei und arbeitet den Graphen manuell, Knoten für Knoten, ab
Stärke	schnell, reproduzierbar	erlaubt Rückfragen und Erläuterungen während der Abarbeitung
Schwäche	keine Rückfrage bei unklaren Stellen; unvollständig, solange es Stubs gibt	langsamer, nicht deterministisch reproduzierbar (abhängig von Modell und Sitzung)
Rolle jetzt	Normalbetrieb für laufende Testläufe	unabhängige Gegenprobe: weichen die Ergebnisse ab, liegt ein Fehler in einem von beiden vor – oder eine unklare Stelle im Regelwerk
Rolle bei Erweiterungen	übernimmt neue oder geänderte Befehle erst danach	neue oder geänderte Befehle/Anweisungen werden zuerst hier durchgespielt

Hinweis Diese Aufgabenteilung gilt modellneutral für jede interpretierende KI, nicht nur für Claude. Bei unklaren Regelwerk-Stellen darf die ausführende KI Rückfragen stellen, statt eigenmächtig zu entscheiden. Die Zahl der Rückfragen hängt vom Modell ab.

1.2 Ein Test im Überblick



Zwischendurch (nur bei Bedarf): Diagnose bei Befehl 164 · Ein-/Ausgabe über Geräte.
Der Lauf endet immer mit einer Endzeile; die Prüfung erfolgt nur nach ordnungsgemäßem Ende.

Abbildung 1: Lebenslauf eines Tests. Rot: Fälle, in denen der Test nicht startet („laut scheitern“).

2 Aufbau einer Testdatei

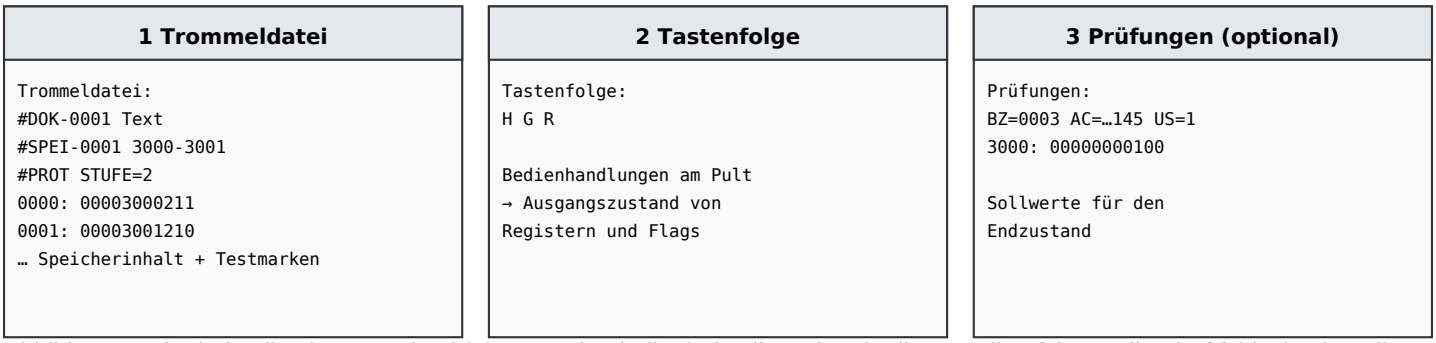


Abbildung 2: Die drei Teile einer Testdatei (eine Textdatei, die drei Teile stehen in dieser Reihenfolge, Teil 3 darf fehlen). Die Zeilen `Trommeldatei:`, `Tastenfolge:` *und* `Prüfungen:` *leiten jeweils einen Teil ein.*

Teil	Pflicht?	Legt fest	Kapitel
Trommeldatei	ja	Anfangsinhalt der Trommel (ein Ausschnitt), die Testmarken (#DOK, #SPEI, #SPUR, #PROT) und die Geräte-Zeilen	3
Tastenfolge	ja	Bedienhandlungen am Pult → Ausgangszustand von Registern und Flags vor Konnektor B	4
Prüfungen	nein	Sollwerte für Register, Flags, Speicherzellen und Ausgabedateien nach Testende	5

Hinweis Dateiname: NNNN-<Operationscodes>.txt für Testdateien in Tests/ (z. B. 0004-210_544_020.txt), asm_NNNN_<Programmname>.txt für Testaufträge aus Assemblerläufen. Das Protokoll trägt denselben Namen mit der Endung .log in Test-Protokolle/.

3 Schritt 1: die Trommeldatei

Der Trommelinhalt wird vor Beginn jedes Tests als Textdatei mit festem Aufbau übermittelt. Er ist nur ein Ausschnitt (adressweise) der Gesamttrommel; nicht genannte Zellen sind 0.

3.1 Zeilenformat

`Adresse: Wert [# Bemerkung]`

- Adresse: 4 Oktalstellen (12 Bit). Wert: 11 Oktalstellen (33 Bit), mit Vornullen ergänzt. Alle Zahlen oktal.
- Leerzeilen werden übergangen. Zeilen mit # sind Kommentar; ein Kommentar darf – durch mindestens ein Leerzeichen getrennt – hinter einer Adresse:Wert-Zeile stehen und gilt ab dem # bis Zeilenende.
- Der Wert besteht aus vier führenden Stellen „beliebig“ (z_0–z_{11}) und sieben Stellen Befehl: 0000 3000 211 = Adresse 3000, Operation 211.

```
# Beginn Programmbefehle
0000: 00003000211
0001: 00003001210
0002: 00000000020
# Beginn Datenbereich
3000: 00000000100 # Startwert der Schleife
3001: 00000000045
```

Achtung Eine nicht genannte Zelle ist 0. Als Befehl gelesen ist 0 der unbedingte Sprung nach Adresse 0000. Programme müssen deshalb mit einem Stopp-Befehl 00000000020 enden, sonst läuft der Test in eine Endlosschleife und bricht nach dem Knotenlimit ab.

3.2 Testmarken

Testmarken sehen aus wie Kommentare, sind aber keine folgenlosen Kommentare. Sie sind eine von der Hardware abweichende Ergänzung, die ausschließlich der besseren Durchführung der Tests dient.

Marke	Bedeutung	Hinweise
#DOK-xxxx <Text>	freier Text zum Test; xxxx oktal. Löst beim Leerbefehl 164 mit der Adresse xxxx die Diagnoseausgabe aus (Kapitel 7).	darf mehrfach mit derselben Nummer vorkommen (Ausgabe in Dateireihenfolge)
#SPEI-xxxx ssss-eeee	gibt zusätzlich den Speicherbereich ssss bis eeee aus	ssss und eeee oktal, gültige Spanne (ssss ≤ eeee); mehrfach zulässig; nach Regelwerk nur zu einem vorhandenen #DOK-xxxx

Marke	Bedeutung	Hinweise
#SPUR-xxxx aaaa	gibt zusätzlich die ganze Spur aus, in der aaaa liegt (alle 32 Zellen, in physischer Zugriffsreihenfolge der Trommel)	welcher Sektor von aaaa gemeint ist, spielt keine Rolle (Bahn-Nr. = aaaa ohne die unteren 5 Bit); mehrfach zulässig
#PROT STUFE=n	Protokolltiefe: 0 / 1 / 2 (Kapitel 8)	höchstens einmal; fehlt sie, gilt Stufe 1

Reihenfolge der Ausgabe zu einem xxxx: zuerst alle #DOK-Texte, danach alle #SPEI-Bereiche, danach alle #SPUR-Spuren, jeweils in Dateireihenfolge.

3.3 Testauftrag aus einem Assemblerlauf

Eine Trommeldatei von Hand zu schreiben ist wegen der oktalen Adressen mühsam. Für Programme in Assemblersprache übernimmt das der Assembler c82ass: Er wandelt ein mit symbolischen Adressen geschriebenes Programm in Oktalbefehle um. Von den erzeugten Dateien wird nur <name>.test benötigt; sie enthält die Trommeldatei im Zeilenformat von 3.1, jede Zeile mit der zugehörigen Quelltextzeile als Bemerkung.

```
2007: 00002006241 # beginn: 241 i ; I:=0
2010: 00002003210 # schleife: 210 azv ; ac:=(azv+i)
```

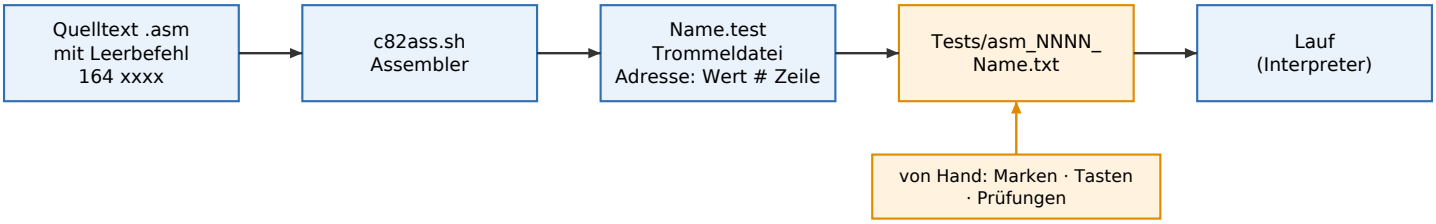


Abbildung 3: Vom Quelltext zum Testauftrag. Von Hand ergänzt werden nur die Debug-Anweisungen, die Startbedingungen und die Prüfungen.

1. Im Assembler-Quelltext an der Stelle, an der ausgewertet werden soll, einen Leerbefehl 164 mit einer vierstelligen Debug-Nummer als Adresse vorsehen, z. B. halt: 164 0001. Die Nummer muss zu den Testmarken #DOK-/#SPEI-/#SPUR- des Testauftrags passen. 2. c82ass.sh -p <Projekt> -s <name> aufrufen. 3. Den Testauftrag Tests/asm_NNNN_<name>.txt anlegen: Zeile Trommeldatei:, die Testmarken (#DOK, #SPEI, #PROT STUFE=n), der Inhalt von <name>.test, dann Tastenfolge: und optional Prüfungen:.

```
Trommeldatei:
#DOK-0001 Test eines mit Assembler assemblierten Programmes
#SPEI-0001 2027-2033
#SPEI-0001 2100-2104
#PROT STUFE=2
0000: 00002000000 # 000 go
2000: 00002007000 # go: 000 beginn
... (weitere Zeilen unverändert aus bsp_5_16.test)
2025: 00000001164 # halt: 164 0001 ;DEBUG Nr. 0001 aktivieren
2026: 00000000020 # 020 0 ;halt
...
Tastenfolge:
H G R
```

Im Beispiel (Quelltext Assembler/Projekt_1/bsp_5.16/bsp_5_16.asm, Test Tests/asm_0001_bsp_5_16.txt) startet H G R bei Adresse 0000; dort steht ein Sprung zum Programmanfang.

3.4 Geräte-Zeilen --leser1, --leser2, --stanzer1, --sw-ausgabe und --k0

Der Testauftrag kann seine Geräte selbst mitbringen: die Lochstreifen für Leser 1 und 2, die Ausgabe des Stanzers, die Schreibwerk-Ausgabe (Kanal K2) und die K0-Eingabequelle. Dazu steht je eine Zeile in der Testdatei, an beliebiger Stelle, üblicherweise bei den Testmarken im Teil Trommeldatei:. Anders als die Testmarken beginnen diese Zeilen nicht mit #, sondern wie die gleichnamigen Schalter des Python-Interpreters mit --. Ein Kommentar # … am Zeilenende wird ignoriert.

Zeile	Bedeutung	Hinweise
--leser1 PFAD	Lochstreifendatei des Lesers 1 (Kanal K1)	Datei oder Ordner; ein Ordner wird in Namensreihenfolge gelesen, für einen Test ist eine einzelne Datei sicherer; höchstens einmal
--leser2 PFAD	Lochstreifendatei des Lesers 2 (Kanal K2)	wie --leser1
--stanzer1 PFAD	Ausgabedatei bzw. Ausgabeordner des Stanzers (Kanal K1)	Datei stanzer-ausgabe.txt; nur bei tatsächlicher Ausgabe angelegt; höchstens einmal

Zeile	Bedeutung	Hinweise
--sw-ausgabe PFAD	jede Ausgabe auf K2 geht in die Datei PFAD/sw-ausgabe.txt statt ans Terminal	die Datei wird nur bei tatsächlicher Ausgabe angelegt und später angehängt; WRZL schließt die aktuelle Zeile ab; höchstens einmal
--k0 datei=PFAD	die Eingaben für K0 (T und SW) kommen aus Datei oder Ordner statt vom Terminal	ein Zeichen oder Name je Zeile; höchstens einmal; der Pfad muss vorhanden sein

- **Vorrang:** Ist derselbe Schalter beim Aufruf angegeben, gilt die Kommandozeile; die Zeile der Testdatei wird übergangen.
- **Relative Pfade** gelten wie auf der Kommandozeile vom aktuellen Ordner des Aufrufs aus, nicht vom Ordner der Testdatei.
- **Formale Prüfung:** Zeile ohne Wert, doppelte Zeile, `--k0` ohne `datei=` und ein nicht vorhandener `--k0`-, `--leser1`- oder `--leser2`-Pfad lassen den Test nicht starten.
- **Claude-Interpreter:** Jede dieser Zeilen gilt als virtuelle Zuordnung bei der Testdefinition.

```
Trommeldatei:
#DOK-0001 Unterprogrammtechnik, Sprung mit Rückkehrabsicht, 025
--sw-ausgabe SW-Ausgabe
#SPEI-0001 2012-2027
#PROT STUFE=2
...
```

Der Lauf dieses Tests (Tests/sw-zeile.txt: drei Unterprogramme geben je ein Zeichen aus) schreibt ABC nach SW-Ausgabe/sw-ausgabe.txt; am Terminal erscheint nichts. Das Beispiel mit echtem Lauf steht in der Bedienungsanleitung, Abschnitt 3.6.

> **Achtung Abweichung Regelwerk ↔ Interpreter (Fassung 04.10.2026):** Das Regelwerk (Durchführung eines Tests, 2b, Stand 04.10.2026) erlaubt alle fünf Zeilenformen. Der Python-Interpreter liest in dieser Fassung als Zeile der Testdatei bisher nur `--sw-ausgabe` und `--k0`; `--leser1`, `--leser2` und `--stanzer1` als Zeile weist die formale Prüfung als ungültiges Zeilenformat ab (Aufruf als Schalter funktioniert). Der Interpreter ist nachzuführen.

4 Schritt 2: die Tastenfolge

Nach der Trommeldatei folgt eine Tastenfolge. Sie wird auf die Ebene von Registern (mit ihren Bits) und Flags (0/1) heruntergebrochen und legt so den Ausgangszustand des Rechners vor dem Eintritt in Konnektor B vollständig fest.

> **Hinweis Prinzip:** Tastenfolgen werden nicht nur als Rohschalterzustände betrachtet, sondern bis zur Wirkung der dadurch geladenen oder ausgelösten Instruktion vorausinterpretiert. Dadurch kann der Automat ab Konnektor B im regulären Vier-Phasen-Zyklus laufen, ohne dass der manuelle Eingriff selbst im Knotengraphen als Sonderfall behandelt werden müsste. Beispiel: G lädt die Instruktion 0000 000 (Sprung nach 0000); dass BZ danach 0000 ist, ist das vorweggenommene Ergebnis dieses Sprungs (⟨BZ⟩ := a, ANWEISUNG-007), nicht die unmittelbare Wirkung von G selbst.

4.1 Beispiel A: Ansprung des Speicherplatzes 0000 (direkt)

Trommeldatei wie in Abschnitt 3.1, Tastenfolge H G R:

Schritt	Aktion	Wirkung auf Register und Flags
1	Schalter H-R → „H“	Flag H := 1
2	Taste „G“	alle Flags außer H, T, BS und SP werden gelöscht, danach US := 1; BR := „0000 000“ (unbedingter Sprung nach Speicherplatz 0000, AR := 0000, OR := 000); K0 angewählt, SW nicht gedrückt → Flag T := 1; ⟨BZ⟩ := 0 als vorweggenommenes Ergebnis des Sprungs
3	Schalter H-R → „R“	Flag H := 0, US := 0; die Interpretation setzt bei Konnektor B fort, mit BZ = 0000

4.2 Beispiel B: Ansprung eines beliebigen Speicherplatzes (hier 0004)

```
# Beginn Programmbefehle
0004: 00003000211
0005: 00003001210
0006: 00000000020
# Beginn Datenbereich
3000: 00000000100
3001: 00000000045

Tastenfolge:
H G M-gedrückt-halten E E E E M-loslassen R
```

Schritt	Aktion	Wirkung auf Register und Flags
1	H-R → „H“	Flag H := 1

Schritt	Aktion	Wirkung auf Register und Flags
2	Taste „G“	alle Flags außer H, T, BS, SP := 0; BR := „0000 000“; K0 angewählt, T := 1
3	„M“ gedrückt halten	OR := OR ∨ 025 → BR := „0000 025“ (Sprung mit Rückkehrabsicht, Adresse 0000)
4	1. „E“	Befehlsentnahme (Phase II) durch gehaltenes M verhindert; BZ := 0001, AR := 0001; ⟨AC⟩ := 0000 0002 000 (Rücksprungadresse BZ+1 im Adressfeld)
5	2. „E“	BZ := 0002, AR := 0002; ⟨AC⟩ := 0000 0003 000
6	3. „E“	BZ := 0003, AR := 0003; ⟨AC⟩ := 0000 0004 000
7	4. „E“	BZ := 0004, AR := 0004; ⟨AC⟩ := 0000 0005 000
8	„M“ loslassen	die OR-Verknüpfung mit 025 entfällt für Folgeschritte
9	H-R → „R“	Flag H := 0; Interpretation bei Konnektor B mit BZ = 0004 → der nächste Befehl wird aus Adresse 0004 entnommen

> **Hinweis** Das Original beschreibt bei diesem Fall als ersten Schritt „Schalter H-R → R“. Das war vermutlich ein Tippfehler: G, M und E sind nur bei Stellung „H“ wirksam. Die Tastenfolge verwendet deshalb „H“; im Bedienpult-Dokument (Abschnitt 4.4) wurde das am 02.10.2026 ebenfalls korrigiert.

Das echte Protokoll (Stufe 2) dieses Beispiels zeigt dieselbe Wirkung. Die Zeilen der Tasten sind vom Interpreter formuliert; der AC steht oktal, elf Stellen:

```
Tastenfolge: H G M-gedrückt-halten E E E E M-loslassen R
  H  : Schalter H-R -> "H", Flag H := 1
  G  : alle Flags ausser H,T,BS,SP := 0 (BS, SP: eingerastete Schalter); BR := 0000 000 (AR:=0000, OR:=000); US := 1;
    K0 angewaehlt, SW nicht gedrueckt -> T := 1; <BZ> := 0
  M gedrueckt halten: OR := OR ODER 025 (BR = AR 025)
  E  : (M gehalten) Befehlsentnahme gesperrt, BZ := 0001, AR := 0001, AC := 00000002000 (BZ+1 im Adressfeld)
  E  : (M gehalten) Befehlsentnahme gesperrt, BZ := 0002, AR := 0002, AC := 00000003000 (BZ+1 im Adressfeld)
  E  : (M gehalten) Befehlsentnahme gesperrt, BZ := 0003, AR := 0003, AC := 00000004000 (BZ+1 im Adressfeld)
  E  : (M gehalten) Befehlsentnahme gesperrt, BZ := 0004, AR := 0004, AC := 00000005000 (BZ+1 im Adressfeld)
  M loslassen: OR-Verknuepfung mit 025 entfaellt fuer Folgeschritte
  R  : Schalter H-R -> "R", Flag H := 0, US := 0; Interpretation beginnt bei Konnektor B
  Nicht spezifizierte Register (AC, ...) starten mit 0.
== 2. Zustaende des Knotenautomaten ==
```

4.3 Festlegungen zu den Tasten M, Ü, E, ST, BS, SP

Taste	Festlegung
E unter gehaltenem M	Jedes E führt Phase IV (BR = ⟨AR⟩ 025: Sprung mit Rückkehrabsicht mit Löschen) und Phase I aus; Phase II ist gesperrt. Nach dem n-ten E (Start mit BZ = b): BZ := b+n, AR := BZ, und der AC enthält im Adressfeld (z_{12}–z_{23}) den Wert BZ+1, übrige Stellen 0 (das Löschbit von 025 ersetzt den AC, es wird nicht aufsummiert). Danach steht der Automat vor Phase II: ein E (nach Loslassen von M) beginnt bei Phase II (Knoten 8), ein R bei Konnektor B.
M	M hat nur zwei Stellungen: „gedrückt“ (M+) und „losgelassen“ (M-). Das Token M allein = M+ und sofort M- (Antippen). Bei M+: OR := OR ∨ 025. Wurde unter M kein E betätigt (Antippen), bleibt BR = „⟨AR⟩ 025“ aufgestellt und wird erst durch E oder R ausgeführt. Wurde mindestens ein E betätigt, entfällt der Beitrag 025 beim Loslassen; übrige Anteile im OR, z. B. von Ü, bleiben.
Ü	nur bei gesetztem H wirksam. AR := 7777 (überschreibt unbedingt), OR := OR ∨ 240; BR = „7777 240“ aufgestellt, noch nicht ausgeführt. Ausgeführt durch E oder R (beides beginnt bei Knoten 19, Ende der Phase-III-Arbeit).
ST	einzige Taste, die nicht an H gebunden ist, dafür elektronisch verriegelt: nur wirksam bei Bedingtem Stopp (BS) oder Unbedingtem Stopp (US). OR := OR ∨ 240; AR unverändert; US := 0; BR aufgestellt (Start bei Knoten 19 durch E oder R). US entsteht in einer Tastenfolge durch G und wird durch ST oder R gelöscht; ohne vorheriges G steht der Rechner zu Beginn nicht im Stopp.
BS, SP	einrastende Schalter. BS=1/SP=1 setzen das gleichnamige Flag auf 1, BS=0/SP=0 auf 0; mehr geschieht nicht. Da die Schalter mechanisch einrasten, lässt G sie unverändert (wie H). Nicht an H gebunden und in jeder Reihenfolge zu den anderen Tasten zulässig.
E (ohne M)	Phase IV und I, dann Phase II und III, danach Halt (Halt-Punkt = Eintritt in Knoten 19). Nur bei gesetztem H wirksam.
Ende der Folge	Eine Tastenfolge darf ohne R enden; dann bleibt der Rechner im Halt, und das Ergebnis ist der Zustand nach dem letzten Schritt.

4.4 Wo der Lauf nach „R“ beginnt

Zustand nach der Tastenfolge	Startpunkt
kein Befehl in BR aufgestellt (z. B. H G R)	Konnektor B (Knoten 1) mit dem Wert in BZ; Phase I lädt den Befehl

Zustand nach der Tastenfolge	Startpunkt
BR aufgestellt, noch nicht ausgeführt (nach Ü, ST, M ohne E oder nach E ohne M)	Knoten 19 (Ende der Phase-III-Arbeit); Phase IV führt BR aus
Tastenfolge endet im Halt (kein R)	es wird nichts mehr ausgeführt; Ergebnis = Zustand nach dem letzten Token

4.5 Weitere typische Bedienfolgen (Fälle 1 bis 6)

Die folgenden Folgen sind unabhängig voneinander: Jede führt nur die Wirkung ihrer eigenen, tatsächlich aufgeführten Tasten aus. Es gibt keinen gemeinsamen Vor-Zustand; nicht genannte Register und Flags bleiben unberührt. Sie dienen der Illustration, sie gehören nicht zum Teststart.

Fall	Folge	Wirkung
1	H G	Flags außer H, T, BS, SP := 0 (damit auch FÜ, FE gelöscht), US := 1; BR := „0000 000“; K0, T := 1; BZ unverändert. Ohne R bleibt der Automat im Halt – das Programm ist unterbrochen.
2	ST	nur im Stopp (BS oder US) wirksam: OR := OR ∨ 240, AR bleibt, US := 0; BR aufgestellt, Start bei Knoten 19 durch E oder R. H bleibt unberührt.
3	H Ü R	Ü wirkt, weil H gesetzt ist: AR := 7777, BR := „7777 (OR ∨ 240)“. R: H := 0, Fortsetzung bei Phase IV → Ausführung von BR.
4	H M R	M (Antippen): OR := OR ∨ 025 → BR := „⟨AR⟩ (OR ∨ 025)“; kein E unter M, BR bleibt aufgestellt. R: Fortsetzung bei Knoten 19.
5	H G Ü E	Ergebnis: ⟨7777⟩ := ⟨AC⟩ (AC in Zelle 7777 transportiert). E (einzelner Tipp): Phase IV führt „7777 240“ aus, dann Phase I, II, III; danach Halt (H bleibt 1) mit neu geladenem BR aus Zelle ⟨BZ⟩.
6	H G M+ E M- E	Ergebnis: ⟨AC⟩ := ⟨BZ⟩+2. Das erste E (unter M) führt Phase IV aus; Phase II bleibt gesperrt. Nach M-Loslassen führt das zweite E Phase II und III aus; danach Halt. Zusatzregel: Mehrfaches erstes E erhöht die im AC abgelegte Adresse je um 1 (zweimal: ⟨BZ⟩+3, und Zelle 0002 wird als nächster Befehl aufgestellt).

Übertragung auf BS, FÜ und FE: BS verhält sich wie US, alle vier Fälle sind anwendbar; zusätzlich muss der Schalter BS gelöst werden (BS=0). FÜ: nur Fall 1; dazu ein eigener Fall „H; E gedrückt halten; G; E loslassen“, der nur den Überlauf löscht statt aller Flags und BR. FE: nur durch G aufhebbar (Fall 1).

> **Achtung** Die Zuordnung „US/BS wird übergangen“ aus dem Bedienpult-Dokument ist keine aus der Tastenfolge ableitbare Tatsache, sondern gilt nur unter der Zusatzannahme, dass AR/OR zufällig gerade den US-Befehl enthalten. Das gehört zur Anwendung, nicht zur Definition der Sequenzen. Ob die Hardware nach „H, G“ wirklich US = 1 anzeigt, ist in der Originaldokumentation nicht belegt und bleibt offen; wirkungslos ist es in jedem Fall, weil „R“ US unabhängig von seinem Wert überwindet.

5 Schritt 3 (optional): Prüfungen

Nach der Tastenfolge kann ein dritter Abschnitt `Prüfungen:` folgen. Fehlt er, endet der Test ohne automatische Ergebnisprüfung. Ist er vorhanden, vergleicht der Interpreter nach der interpretativen Abarbeitung den Endzustand mit den Sollwerten und vermerkt Test wahr oder Test falsch. Dadurch wird jeder Testlauf zum Regressionstest: Ein neuer Lauf wird gegen die zuletzt bestätigten Sollwerte verglichen, ohne dass ein Mensch das Protokoll erneut lesen muss.

5.1 Aufbau

Teil	Inhalt
1. Register-/Flag-Zeile (höchstens eine)	Zuweisungen SCHLÜSSEL=WERT, durch Leerzeichen getrennt, beliebige Reihenfolge. Schlüssel: BZ, AC, AR, FE, FUE (für FÜ), BS, US, E, A, V1, V2 – genau die Teilmenge, die auch die Endzeile des Interpreters ausgibt. Nicht vorgesehen: OR, H, SP, T, SW, K1–K3. Die Zeile muss nicht vollständig sein; jeder Schlüssel höchstens einmal. BZ= und AR=: 4 Oktalstellen · AC=: 11 Oktalstellen · Flags: genau 0 oder 1.
2. Speicherprüfzeilen (beliebig viele)	je eine pro Zeile im Format Adresse: Wert (4 + 11 Oktalstellen): Inhalt, den die Trommeladresse nach Testende haben muss. Dieselbe Adresse darf nicht mit widersprüchlichem Sollwert auftreten.
3. Dateiprüfungen (höchstens zwei)	je eine Zeile `--pruefe-stanzer1 DATEI` bzw. `--pruefe-sw-ausgabe DATEI` (Abschnitt 5.4).

Fehlt die Register-/Flag-Zeile, beginnt der Abschnitt unmittelbar mit Speicherprüfzeilen. Ein leerer Abschnitt Prüfungen: gilt wie ein fehlender; ein Abschnitt, der nur Dateiprüfungen enthält, ist gültig.

```
Prüfungen:
BZ=0004 AC=00000000145 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
3000: 00000000100
```

```
3001: 00000000045
```

5.2 Auswertung und Ausgabe

- Die Prüfung erfolgt erst nach Abschluss der Abarbeitung, gegen den dann erreichten Endzustand.
- Jede angegebene Zuweisung und jede Speicherzeile wird einzeln verglichen; zu jeder wird Soll und Ist ausgegeben, damit im Fehlerfall sofort sichtbar ist, welche Prüfung abweicht.
- Test wahr gibt es nur, wenn alle Einzelprüfungen stimmen (einschließlich der Dateiprüfungen nach 5.4); sobald mindestens eine abweicht, ist das Ergebnis Test falsch.
- Die Ausgabe erfolgt stufenunabhängig auf dem Bildschirm (wie die Diagnose zu Befehl 164); bei Protokollstufe 2 zusätzlich am Ende der Protokolldatei, im Anschluss an die Endzustandszeile.
- Test ohne ordnungsgemäßes Ende: Bricht die Abarbeitung vorher ab (Stub, Laufzeitfehler, Knotenlimit), wird keine Prüfung vorgenommen und stattdessen vermerkt, dass sie entfällt. Ein solcher Fall gilt nicht automatisch als Test falsch.

Beispiel: derselbe Lauf, einmal mit richtigem und einmal mit falschem Sollwert (BZ = 0003 statt 0004):

```
$ python3 c8205_interp.py Tests/pruef-wahr.txt
=== Test pruef-wahr (#PROT STUFE=0) ===
-- Diagnose --
#DOK-0001 Test Ladebefehl und Addition
Ende: Unbedingter Stop (US:=1) | Knoten: 90 | BZ=0004 AC=00000000145 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
Ergebnispruefung:
  BZ: soll=0004 ist=0004 OK
  AC: soll=00000000145 ist=00000000145 OK
  AR: soll=0000 ist=0000 OK
  FE: soll=0 ist=0 OK
  FUE: soll=0 ist=0 OK
  BS: soll=0 ist=0 OK
  US: soll=1 ist=1 OK
  E: soll=0 ist=0 OK
  A: soll=0 ist=0 OK
  V1: soll=0 ist=0 OK
  V2: soll=0 ist=0 OK
  3000: soll=00000000100 ist=00000000100 OK
  3001: soll=00000000045 ist=00000000045 OK
Test wahr

$ python3 c8205_interp.py Tests/pruef-falsch.txt
=== Test pruef-falsch (#PROT STUFE=0) ===
-- Diagnose --
#DOK-0001 Test Ladebefehl und Addition
Ende: Unbedingter Stop (US:=1) | Knoten: 90 | BZ=0004 AC=00000000145 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
Ergebnispruefung:
  BZ: soll=0003 ist=0004 FEHLER
  AC: soll=00000000145 ist=00000000145 OK
  ... (übrige Zeilen wie oben, alle OK)
Test falsch
```

5.3 Typische Sollwerte

Wenn der Test …	prüfen Sie
mit einem Stopp-Befehl endet	US=1, BZ= (um 1 hinter dem Stopp-Befehl), AC= mit dem erwarteten Ergebnis
Zahlen in den Speicher schreibt	die Zielzellen als Speicherprüfzeilen; Vorsicht bei Gruppenbefehlen: sie überschreiben bis zu 32 Zellen einer Spur
einen Überlauf erwartet	FUE=1
eine Eingabe liest	E=0, FE=0 und den AC; mit leerem Leser: E=1
Vergleichersprünge benutzt	V1=, V2= (V3 = beide 1)
auf Stanzer oder Schreibwerk ausgibt	die Dateiprüfung nach 5.4

5.4 Dateiprüfungen für Stanzer und Schreibwerk

Programme, die auf Kanal K1 (Lochstreifenstanzer) oder K2 (Schreibwerk) ausgeben, erzeugen eine Ausgabedatei (`stanzer-ausgabe.txt` bzw. `sw-ausgabe.txt`). Zwei weitere, ebenfalls optionale Prüfungen vergleichen diese Dateien mit je einer Soll-Datei. Sie stehen als **Zeile im Abschnitt** `Prüfungen:` der Testdatei (je Zeile eine Prüfung, jede höchstens einmal); der Python-Interpreter nimmt sie zusätzlich als Schalter der Kommandozeile entgegen, und ein Schalter hat Vorrang vor der Zeile der Testdatei. Beim Claude-Interpreter gilt die entsprechende virtuelle Zuordnung bei der Testdefinition.

Zeile / Schalter	Soll-Datei wird verglichen mit
--pruefe-stanzer1 DATEI	der Datei, die der Lauf über `--stanzer1 PFAD` (Schalter oder Zeile der Testdatei, 3.4) erzeugt hat (bei einem Verzeichnis: stanzer-ausgabe.txt darin)
--pruefe-sw-ausgabe DATEI	der Datei, die der Lauf über `--sw-ausgabe PFAD` (Schalter oder Zeile der Testdatei, 3.4) erzeugt hat (bei einem Verzeichnis: sw-ausgabe.txt darin)

Beide Prüfungen können einzeln oder gemeinsam angegeben werden, auch zusammen mit Register-, Flag- und Speicherprüfungen.

- **Soll-Datei:** die hinter dem Schalter genannte DATEI. Ein reiner Dateiname ohne Verzeichnis liegt im Verzeichnis der Testdatei (üblicherweise Tests/); eine Angabe mit Verzeichnis gilt, wie sie geschrieben ist.
- **Ist-Datei:** die vom Lauf erzeugte Ausgabedatei. Maßgeblich ist nur eine Datei, die dieser Lauf angelegt hat; eine übrig gebliebene Datei aus einem früheren Lauf gilt als nicht vorhanden, wenn der Lauf selbst keine Ausgabe auf dem Gerät vorgenommen hat.
- **Prüfung falsch:** Soll-Datei fehlt, Ist-Datei fehlt (auch beide) oder der Inhalt unterscheidet sich. Verglichen wird zeilenweise wie bei diff; Unterschiede nur im Zeilenende (CR LF gegen LF) oder ein fehlendes Zeilenende nach der letzten Zeile zählen nicht, Leerzeichen am Zeilenende dagegen schon.
- **Prüfung wahr:** beide Dateien vorhanden und zeilenweise gleich.
- **Gesamtergebnis:** Test wahr nur, wenn jede Einzelprüfung nach 5.1 und jede Dateiprüfung wahr ist. Wie bei 5.2 entfallen die Dateiprüfungen, wenn der Test nicht ordnungsgemäß endet.
- **Ausgabe:** je Dateiprüfung eine Zeile mit Soll- und Ist-Pfad und OK bzw. FEHLER samt Grund (Soll-Datei fehlt, Ist-Datei fehlt, vom Lauf nicht angelegt, Inhalt verschieden); bei verschiedenem Inhalt folgt der diff (Soll gegen Ist, höchstens 100 Zeilen).
- **Formale Prüfung:** `--pruefe-stanzer1` ist nur zusammen mit `--stanzer1`, `--pruefe-sw-ausgabe` nur zusammen mit `--sw-ausgabe` zulässig (Schalter oder Zeile der Testdatei), weil sonst keine Ist-Datei existieren kann. Als Schalter der Kommandozeile sind beide nur bei einer Testdatei je Aufruf zulässig. Eine fehlende Soll-Datei ist kein Formfehler, sondern das Prüfergebnis falsch.

Beispiel (echter Lauf): Das Programm wählt K1 an (`4600 030`) und stanzt drei Zeichen; die Soll-Datei `Tests/stz-soll.txt` enthält die Zeilen 00110001, 00110010 und 00100011. Ende der Testdatei Tests/stz.txt:

```
Prüfungen:
--pruefe-stanzer1 stz-soll.txt
```

```
$ python3 c8205_interp.py --stanzer1 Stanzer1 Tests/stz.txt
=== Test stz (#PROT STUFE=0) ===
-- Diagnose --
#DOK-0001 Stanzer: drei Zeichen
Ende: Unbedingter Stop (US:=1) | Knoten: 117 | BZ=0006 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
Ergebnispruefung:
  Stanzer 1: soll=Tests/stz-soll.txt ist=Stanzer1/stanzer-ausgabe.txt OK
Test wahr
```

Rückkehrcode 0 bei Test wahr, 6 bei Test falsch.

6 Formale Gültigkeitsprüfung vor dem Start

Vor jedem Test werden Trommeldatei, Tastenfolge, Prüfungen und Testmarken formal geprüft. Der Test startet nur, wenn die Prüfung insgesamt positiv verläuft. Bei einer Unstimmigkeit darf er nicht eigenmächtig korrigiert und gestartet werden; die Durchführung wartet auf eine Klärung mit dem Auftraggeber.

Bereich	Geprüft wird
Trommeldatei	Zeilenformat (Adresse 4, Wert 11 Oktalstellen, nur Oktalzahlen, zulässige Kommentar- und Leerzeilen); keine Adresse mit widersprüchlichem Wert mehrfach belegt
#SPEI-xxxx	nach Regelwerk: nur zu einem im selben Test definierten #DOK-xxxx gleicher Nummer; ssss-eeee bildet eine gültige Spanne
#PROT STUFE=n	n ist 0, 1 oder 2; höchstens einmal; Fehlen ist gültig (Standard: 1)
Tastenfolge	ergibt einen vollständig definierten Ausgangszustand. Ungültig: M- ohne vorheriges M+, zweites M+ bei gedrücktem M, R oder Ende bei gedrücktem M, unbekannte Tasten, Ende weder mit R noch im Halt. Gültig, aber ohne Wirkung: Ü und E bei nicht gesetztem H.
Geräte-Zeilen	--leser1, --leser2, --stanzer1, --sw-ausgabe, --k0: Zeile mit Wert; jede höchstens einmal; --k0 in der Form datei=PFAD; die Pfade von --k0, --leser1 und --leser2 müssen vorhanden sein (3.4)
Prüfungen:	unbekannte Schlüssel, falsche Stellenzahl oder unzulässiger Wert, mehrfach genannter Schlüssel, Zeile außerhalb des Adresse:Wert-Formats, widersprüchlich mehrfach belegte Prüfadresse
Dateiprüfungen	--pruefe-stanzer1 setzt --stanzer1, --pruefe-sw-ausgabe setzt --sw-ausgabe voraus; höchstens eine Zeile je Prüfung; als Schalter nur bei einer Testdatei je Aufruf (5.4)

Achtung Abweichung Regelwerk ↔ Interpreter (Fassung 04.10.2026): Der Python-Interpreter prüft nicht, ob zu jedem #SPEI ein #DOK gleicher Nummer existiert; er gibt die Zellen auch ohne #DOK aus (Interpreter-Dokument: `#SPEI` und `#SPUR` brauchen kein `#DOK`). Ob das Regelwerk oder der Interpreter anzupassen ist, ist nicht entschieden.

Hinweis Grundsatz „laut scheitern“ (gilt ebenso später bei der Abarbeitung des Knotengraphen): Ergibt sich eine Unklarheit, ein Widerspruch oder eine fehlende Festlegung, wird das laut gemeldet und der Test angehalten, statt eigenmächtig eine Annahme zu treffen. Das gilt unabhängig davon, ob Python-Interpreter oder KI den Test ausführen.

7 Diagnose beim Leerbefehl 164

Die Diagnoseausgabe gibt während des Laufs Zwischenstände aus. Sie ist, wie die Marken #DOK und #SPEI, eine von der Hardware abweichende Zusatzdefinition ohne Entsprechung in der Elektronik.

7.1 Auslösebedingung

Tritt in der Befehlsfolge der Befehl 164 (oktal; Leerbefehl, B = 1, A = 6, Z = 4) mit einer Adresse a auf und ist für xxxx = a eine Testmarke #DOK-xxxx definiert, wird zusätzlich zur regulären Abarbeitung die Diagnose ausgegeben.

7.2 Ablauf

1. Der Befehl 164 wird zuerst genau so interpretiert, wie es der Knotenautomat vorgibt; die Ausführung selbst erfährt keine Sonderbehandlung. 2. Danach wird ausgegeben: die Testmarke #DOK-xxxx mit ihrem Text. 3. Ist zu xxxx ein #SPEI-xxxx ssss-eeee definiert: alle Zellen ssss bis eeee zeilenweise Adresse: Wert. 4. Ist ein #SPUR-xxxx aaaa definiert: danach, nach allen #SPEI-Ausgaben, alle 32 Zellen der Spur in physischer Zugriffsreihenfolge, Format wie bei #SPEI. 5. Danach wird die Interpretation regulär fortgesetzt.

Die Diagnose erscheint stufenunabhängig immer auf dem Bildschirm; bei Stufe 2 zusätzlich in der Protokolldatei. Nach dem Regelwerk gehören zur Textzeile auch BZ, AC und die Flags FE, FÜ, BS, US, E, A; der Python-Interpreter gibt diese Zeile nicht aus (Änderung 29.09.2026: es stehen Marke, Text und Speicherzeilen da).

Beispiel mit #SPUR: Der Test hält bei Befehl 164 (Adresse 0020) und gibt die Spur der Zelle 4001 aus. Die Zellen erscheinen in der Reihenfolge der Trommel, nicht aufsteigend (vgl. Tabelle in der Rechnerbeschreibung, Kapitel 6.1):

```
$ python3 c8205_interp.py Tests/spur.txt
=== Test spur (#PROT STUFE=0) ===
-- Diagnose --
#DOK-0020 x
#SPUR-0020 4001
4000: 00000000000
4015: 00000000000
4032: 00000000000
4007: 00000000000
4024: 00000000000
4001: 00000000000
4016: 00000000000
4033: 00000000000
… (28 weitere Zellen, alle 0, in Trommelreihenfolge) …
Ende: Unbedingter Stop (US:=1) | Knoten: 41 | BZ=0002 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
```

Zelle für Zelle: 4000, 4015, 4032, 4007, 4024, 4001, … – das ist die Spalte „logische Sektoradresse“ der physischen Reihenfolge (Interleaving).

Hinweis Die Leerbefehle 164 und 166 werden ohne Resultatbehandlung ausgeführt (das Z-Feld bewirkt keine Verschiebung des AC). Es ist daher unschädlich, die Marke an eine beliebige Programmstelle zu setzen, und sie eignet sich zum Messpunkt zwischen zwei Schritten.

8 Protokoll und Protokolltiefe

#PROT STUFE=	Knotenprotokoll	Protokolldatei	Diagnose (164) und Prüfung
0	keines	keine	immer auf dem Bildschirm
1 (Standard)	auf dem Bildschirm, knapp: eine Zeile je durchlaufenem Knoten	keine	immer auf dem Bildschirm
2	in der Protokolldatei	ja (für den ganzen Test, auch ohne Ausgabebefehle)	Bildschirm und Datei

Die Stufen sind nützlich vor allem bei Programmen mit vielen Durchläufen (z. B. Schleifen über Gruppen- oder Wiederholungsbefehle), bei denen ein Protokoll jedes einzelnen Knotens unübersichtlich lang würde. Der Python-Interpreter nimmt die Knotenzeilen nur auf, wenn er mit --knotentest aufgerufen wird (Bedienungsanleitung,

Abschnitt 6.4).

8.1 Inhalt der Protokolldatei (Stufe 2)

1. Ausgangsbedingungen: die übernommene Trommeldatei und die Tastenfolge mit ihrer Wirkung auf Register und Flags. 2. Zustände des Knotenautomaten: die ab Konnektor B erreichten Knoten in der durchlaufenen Reihenfolge, mit BZ, AR und AC. 3. Debug-Ausgaben nach Abschnitt 7 (#DOK, #SPEI, #SPUR). 4. Ende: Endgrund und Endzeile; danach, falls ein Prüfungen:-Abschnitt vorhanden ist, die Ergebnisprüfung mit Soll und Ist je Wert (einschließlich der Dateiprüfungen nach 5.4) und dem Gesamtergebnis.

Zum Lesen ein echter Protokollauszug (Beispiel A aus Abschnitt 4.1: Laden, Addieren, Stopp; die Marke #PROT STUFE=2 und --knotentest waren gesetzt):

```
Test bspA
== 1. Ausgangsbedingungen ==
Trommeldatei (uebernommen, Ausschnitt; nicht genannte Zellen = 0):
  0000: 00003000211
  0001: 00003001210
…
== 2. Zustaende des Knotenautomaten ==
K001 KON B BZ=0000 AR=0000 AC=00000000000
  BR=0000 000  (vor Knoten 002)  BZ=0000 AC=00000000000 FÜ=0 V1=0 V2=0 V3=0
K002 PHA Phase I BZ=0000 AR=0000 AC=00000000000
K003 ANW 001 BZ=0001 AR=0000 AC=00000000000
K004 TST 002 BZ=0001 AR=0000 AC=00000000000 -> falsch
K005 TST 003 BZ=0001 AR=0000 AC=00000000000 -> wahr
K008 PHA Phase II BZ=0001 AR=0000 AC=00000000000
K009 ANW 005 BZ=0001 AR=0000 AC=00000000000
K010 TST 006 BZ=0001 AR=0000 AC=00000000000 -> falsch
K011 ANW 007 BZ=3000 AR=0000 AC=00000000000
K012 TST 008 BZ=3000 AR=0000 AC=00000000000 -> falsch
K015 PHA Phase III BZ=3000 AR=0000 AC=00000000000
K016 ANW 001 BZ=0001 AR=3000 AC=00000000000
K017 TST 011 BZ=0001 AR=3000 AC=00000000000 -> wahr
…
K045 ANW 030 BZ=0003 AR=0000 AC=00000000145
== Ende ==
Unbedingter Stop (US:=1)
BZ=0003 AC=00000000145 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
```

8.2 Eine Knotenzeile lesen

Zeile	Lesart
K001 KON B BZ=0000 AR=0000 AC=…	Knoten 1 (Konnektor B): Eintritt in den Ablauf. Danach BZ, AR, AC oktal.
BR=0000 000 (vor Knoten 002) …	Befehlsregister (Adressteil und Operationsteil) vor Beginn von Phase I; dazu BZ, AC, FÜ, V1, V2, V3
K002 PHA Phase I …	Phasenzuordnung: Beginn der Phase I (analog Phase II: Knoten 8, III: 15, IV: 31/61/67)
K003 ANW 001 BZ=0001 AR=0000 …	Knoten 3 ist eine Anweisung, ANWEISUNG-001: AR := BZ und gleichzeitig BZ := AR + 1 mit den alten Werten. BZ=0001 ist deshalb richtig (nicht 0000+1 mit neuem AR).
K004 TST 002 … -> falsch	Knoten 4, ein Test (ANWEISUNG-002, „X-Abbruch-Merker gesetzt?“): Ergebnis falsch → weiter über den falsch-Ausgang
K016 ANW 001 …	derselbe Anweisungs-Typ in Phase III (Knoten 16): AR wird auf die Operandenadresse gestellt; BZ := alter AR + 1 = BADR + 1
K045 ANW 030 …	ANWEISUNG-030, unbedingter Stopp: Endknoten, US := 1

> **Hinweis** Die „Knoten“-Zahl der Endzeile und die Zahl der Knotenzeilen sind fassungsabhängig und keine Zeitangabe (alle Zeiten sind 0). Sie eignen sich nicht als Sollwert eines Tests.

9 Ein- und Ausgabe in Tests

Programme können Zeichen ein- und ausgeben. Im Test werden die Geräte nur mit den Kanal- und Adresszuordnungen des Originals nachgebildet, ohne mechanische Bauteile. Jede virtuelle Datei wird nur angelegt, wenn der Test das Gerät benutzt.

Kanal	Eingabe (Regelwerk)	Ausgabe (Regelwerk)
K0 mit T oder SW	über die Host-Tastatur: pro Eingabebefehl genau ein Zeichen wird angefordert (nicht eine Zeile); es wird nach der Kodetabelle in die Eingabeverschlüsselung umgesetzt und mit dem Adressregister ODER-verknüpft; danach wird der Eingabebefehl umgestellt	—

Kanal	Eingabe (Regelwerk)	Ausgabe (Regelwerk)
K1	Leser 1: virtueller Lochstreifen (Textdatei), pro Eingabebefehl eine Zeile aus acht Zeichen 0/1 (Spuren w v u p 8 4 2 1; Tr wird nicht dargestellt)	Stanzer 1: Datei, ein ausgegebenes Zeichen je Zeile, acht 0/1 in derselben Spurreihenfolge
K2	Leser 2: wie Leser 1, aber getrennte Lochstreifendatei	Schreibwerk: zeichenweise Textausgabe; WRZL beendet die Zeile (mehrere WRZL = Leerzeilen)

9.1 Lochstreifendateien

- Für Leser 1 und Leser 2 werden stets getrennte Dateien benutzt. Jeder Leser muss durch `--leser1` bzw. `--leser2` (Schalter oder Zeile der Testdatei, 3.4) definiert werden; fehlt die Zuordnung, ist auf dem Kanal keine Eingabe möglich.
- Ist die aktuelle Datei zu Ende und tritt für denselben Leser ein weiterer Eingabebefehl auf, wird eine neue Lochstreifendatei angefordert; der Python-Interpreter wartet dann still (ohne Meldung), bis eine neue Datei erscheint.
- Wird eine Datei nicht vollständig verbraucht, bleibt sie für den Leser geladen und wird bei einem späteren Eingabebefehl an der zuletzt entnommenen Stelle fortgesetzt.
- Geht der Rechner in den Halt (Stopp, Fehler-Halt oder sonst), werden alle angebrochenen Dateien beider Leser beendet und entfernt; ein neuer Test fordert seine Dateien neu an.
- Eine Zeile mit gerader Zahl von Einsen ist ein Fehler bei Eingabe (FE := 1, der Lauf hält). Leerzeilen sind Vorspann und werden übergangen. Das Lochbild 11111111 („Irrung Zeichen“) wird übersprungen: keine Paritätsprüfung, kein FE, nicht ins AR; der Eingabebefehl liest das nächste Zeichen (Interpreter-Dokument 8.2, Ergänzung 04.10.2026).

9.2 Schreibwerk

Das Regelwerk beschreibt eine Ausgabe in die Datei sw-ausgabe.txt (höchstens einmal je Test angelegt, nur bei tatsächlicher Ausgabe, später angehängt statt überschrieben, am Testende zum Download bereitgestellt). Ist dem Test die Datei zugeordnet (Schalter --sw-ausgabe PFAD oder gleichnamige Zeile in der Testdatei, 3.4), geht jede Ausgabe auf K2 dorthin; fehlt die Zuordnung, erscheint sie auf dem Terminal (Dunkelgelb). Beides zugleich gibt es nie. Die Ausgabe ist vom Schalter SW unabhängig; ein Warten auf SW findet nicht statt.

> **Hinweis** Die Schreibwerk-Ausgabe über K2 ist unabhängig vom Schalter SW (Hardwarebeschreibung und Regelwerk 6.3.1); der Schalter SW betrifft nur die Eingabeseite (K0). Für den Stanzer gibt es dagegen keinen Terminal-Ausweg: ohne `--stanzer1` unterbleibt die Ausgabe auf K1 ganz.

9.3 Gegenüberstellung der Ausführenden

	Python-Interpreter	Claude-Interpreter
K0-Eingabe	Terminal (ein Tastendruck je Eingabebefehl) oder --k0 datei=PFAD (Schalter oder Zeile der Testdatei)	die KI fordert pro Eingabebefehl ein Zeichen auf dem Host-Bildschirm an und nimmt es entgegen
Leser 1/2	--leser1, --leser2 (Datei oder Ordner; Schalter, nach Regelwerk auch Zeile der Testdatei)	die KI fordert die Lochstreifendatei für den jeweiligen Leser an
Stanzer 1	--stanzer1 PFAD (Schalter, nach Regelwerk auch Zeile der Testdatei)	Datei stanzer-ausgabe.txt, am Testende zum Download
Schreibwerk	--sw-ausgabe PFAD (Schalter oder Zeile der Testdatei) → Datei sw-ausgabe.txt; sonst Terminal, Dunkelgelb	Datei sw-ausgabe.txt, am Testende zum Download
Protokoll	--out …, bei Stufe 2 Datei	Datei protokoll.txt, am Testende zum Download

10 Den Claude-Interpreter einsetzen

Der Claude-Interpreter ist ein Verfahren, kein Programm: Eine KI führt den Knotengraphen von Hand aus. Es ist kein Ersatz für den Python-Interpreter, sondern eine unabhängige Gegenprobe und das Werkzeug für neue oder geänderte Befehle.

10.1 Wann?

- Zur Gegenprobe, solange der Python-Interpreter nicht vollständig und stubfrei ist: Ergebnis und Protokoll beider Wege vergleichen.
- Bei einer Rückfrage zu einer Ableitung oder bei einer unklaren Stelle im Regelwerk.
- Beim Durchspielen neuer oder geänderter Befehle/Anweisungen, bevor sie in c8205_interp.py übernommen werden.

10.2 Was die KI braucht

Material	Zweck
Knotengraph (Eingänge_der_Knoten.md) und Variablentabelle T2 (Beschreibung_der_Knoten.md)	der Ablauf selbst und die Bedeutung jeder ANWEISUNG
Befehlsliste (Befehlsliste.md)	die Formel je Operationscode (ANWEISUNG-021 verweist dorthin)
Datenstruktur, Magnettrommel, Bedienpult, Kodetabellen	Register, Felder, Flags; Sektorreihenfolge; Tasten; Zeichen
Diese Testdefinition (Durchführung_eines_Tests.md) und die Testdatei	Format, Tastenfolge, Marken, Protokoll
baz-wirkungstabelle.md	Cache bereits hergeleiteter Knotenpfade und Formeln je BAZ-Wert, damit nichts doppelt hergeleitet wird; der Python-Interpreter liest sie nicht

10.3 Vorgehen

1. Testdatei formal prüfen (Kapitel 6). Bei Unstimmigkeit nicht starten, sondern rückfragen. 2. Tastenfolge auf Register und Flags herunterbrechen (Kapitel 4) und den Startpunkt bestimmen (Konnektor B oder Knoten 19). 3. Den Graphen Knoten für Knoten ablaufen: Knotentyp, ANWEISUNG aus T2, Ergebnis des Tests (wahr/falsch) und die neuen Registerwerte festhalten. ANWEISUNG-001 und andere „&&“-Anweisungen mit den Werten vor dem Knoten auswerten. 4. Bei Ein-/Ausgabebefehlen die Geräteanforderungen nach Kapitel 9 stellen (ein Zeichen je Eingabebefehl). 5. Bei Befehl 164 mit passender Marke die Diagnose ausgeben (Kapitel 7). 6. Am Ende die Endzeile und – falls vorhanden – die Ergebnisprüfung angeben; das Protokoll im selben Format wie der Python-Interpreter führen. 7. Bei jeder Unklarheit anhalten und rückfragen („laut scheitern“), nicht annehmen.

> **Hinweis** Weichen Python- und Claude-Ergebnis voneinander ab, ist das kein Grund, eines der beiden anzupassen, sondern ein Befund: ein Fehler im Interpreter, ein Fehler im Claude-Durchlauf oder eine tatsächlich unklare Stelle im Regelwerk. Vergleichen Sie Protokoll und Knotenfolge, bis die erste Abweichung gefunden ist.

11 Referenztests und Regression

Für mehrere Tests sind Referenzläufe festgehalten (u. a. Tests 0004 bis 0021 sowie s1-R1 bis s4-spur, dazu die Lochband-Tests lb-001 bis lb-003). Ein neuer Lauf wird mit den zuletzt bestätigten Sollwerten verglichen: Endzustand von Register und Flags, erwartete Speicherinhalte, erwartete Ausgaben.

- **Vollständigkeitstest:** c8205_befehlssweep.py baut für alle 512 Operationscodes ein Minimalprogramm (der Befehl an Adresse 0000, der Stopp 020 an Adresse 0001) und führt es über dieselbe Knotengraph-Engine aus. Er prüft nicht die numerische Korrektheit jeder Formel, sondern nur, dass für jeden Code eine Implementierung vorhanden ist und der Automat sauber terminiert (kein Stub, kein Sackgassen-Knoten, keine erkennbare Endlosschleife).
- **Formelkorrektheit:** dafür dienen die regulären Tests mit Prüfungen (Kapitel 5), z. B. über c8205_interpreter.sh/.bat.
- Bei einem neuen oder geänderten Befehl zuerst den Claude-Interpreter benutzen (Kapitel 10), dann den Befehl in c8205_interp.py übernehmen, danach alle Referenztests und den Befehlssweep erneut laufen lassen.

```
python3 c8205_befehlssweep.py            # alle 512 Codes, Kurzbericht
python3 c8205_befehlssweep.py --verbose  # jeden Code einzeln
python3 c8205_befehlssweep.py --code 251 # nur ein Code (oktal), im Detail
```

12 Checkliste und Vorlage

12.1 Checkliste für neue Tests

1. Programm endet mit einem Stopp-Befehl (00000000020)? 2. Adressen vierstellig, Werte elfstellig, nur Ziffern 0–7? 3. Keine Adresse doppelt mit verschiedenem Wert? 4. Messpunkt gesetzt (164 xxxx) und passende #DOK/#SPEI/#SPUR vorhanden? 5. #PROT STUFE= bewusst gewählt (Schleifen: 0 oder 2)? 6. Tastenfolge endet mit R oder im Halt; SW=1 vor G, wenn K0 mit Schreibwerk gebraucht wird? 7. Wenn das Programm Zeichen liest oder schreibt: Kanal angewählt und das Gerät zugeordnet (Schalter oder Zeile --leser1, --leser2, --stanzer1, --sw-ausgabe, --k0 in der Testdatei)? 8. Prüfungen: mit den wichtigsten Sollwerten, damit der Test als Regressionstest taugt; bei Stanzer oder Schreibwerk zusätzlich die Dateiprüfung (--pruefe-stanzer1, --pruefe-sw-ausgabe)?

12.2 Vorlage

Die Vorlage ist mit dem Python-Interpreter lauffähig und entspricht dem Test aus Kapitel 5; bei Stufe 2 entsteht das Protokoll nach Test-Protokolle/…log.

```
Trommeldatei:
#DOK-0001 Kurzbeschreibung des Tests
#SPEI-0001 3000-3001
#PROT STUFE=2
# Programm
0000: 00003000211 # AC := <3000>
0001: 00003001210 # AC := AC + <3001>
0002: 00000001164 # Leerbefehl 164 -> Diagnose #DOK-0001
0003: 00000000020 # Stopp
# Daten
3000: 00000000100
3001: 00000000045

Tastenfolge:
H G R

Prüfungen:
BZ=0004 AC=00000000145 US=1
3001: 00000000045
```

Anhang A: Begriffe

Begriff	Erklärung
Test	ein simulierter Rechnerlauf: Trommeldatei + Tastenfolge (+ Prüfungen)
Trommeldatei	Textdatei mit dem Speicherausschnitt, mit dem ein Test beginnt, den Testmarken und den Geräte-Zeilen
Tastenfolge	Bedienhandlungen am Pult, die den Ausgangszustand von Registern und Flags festlegen
Testmarke	Zeile mit #, die nur der Testdurchführung dient (#DOK, #SPEI, #SPUR, #PROT)
Geräte-Zeile	Zeile der Testdatei mit -- (--leser1, --leser2, --stanzer1, --sw-ausgabe, --k0), die ein virtuelles Gerät zuordnet
Dateiprüfung	Prüfung, die die Stanzer- oder Schreibwerk-Ausgabedatei mit einer Soll-Datei vergleicht (--pruefe-stanzer1, --pruefe-sw-ausgabe)
Leerbefehl 164	Operation B = 1, A = 6, Z = 4: tut nichts (Messpunkt); löst bei passender Adresse die Diagnose aus
Konnektor B	Eintritt in den Ablauf von außen (Knoten 1 des Knotengraphen)
Knoten 19	Ende der Phase-III-Arbeit; Startpunkt bei aufgestelltem Befehlsregister (nach Ü, ST, M ohne E oder E ohne M)
Knotenautomat	der Knotengraph des Befehlsablaufs mit seinen vier Knotentypen (Konnektor, Phasenzuordnung, Anweisung, Test)
Stub	Meldung „an dieser Stelle fehlt eine Umsetzung“; der Test wird angehalten
Regressionstest	ein Test, dessen Sollwerte bestätigt sind und der bei jeder Änderung erneut gegen sie verglichen wird
Claude-Interpreter	Verfahren, bei dem eine KI den Knotengraphen von Hand abarbeitet (Gegenprobe, neue Befehle)
Interleaving	logische Reihenfolge der Sektoren einer Spur ≠ physische; Schritt +05 (oktal) von logisch L nach L+1

Anhang B: Verweise

Thema	Dokument
Aufruf, Schalter, Fehlermeldungen des Interpreters	PDF „Bedienungsanleitung Python-Interpreter“
Rechner, Phasen, Register, Trommel	PDF „Rechnerbeschreibung“
Tasten, Schalter, Geräte	PDF „Bedienpult und Peripherie“
Alle Operationscodes	PDF „Einzelbefehlsliste“
Zeichenkodes (Tabelle 1)	PDF „Kodetabellen“
Knotengraph und T2	PDF „Knotengraph“
Assembler, .test-Datei	PDF „Assembler-Dokumentation“